

Covert tree to BST

```
// Inorder traversal to store the nodes in a vector
void inorder(Node* root, vector<int>& nodes) {
    if (root == nullptr) {
        return;
    }
    inorder(root->left, nodes);
    nodes.push_back(root->data);
    inorder(root->right, nodes);
}

// Inorder traversal to convert tree
// to BST.
void constructBST(Node* root, vector<int> nodes, int& index) {
    if (root == nullptr) return;

    constructBST(root->left, nodes, index);

    // Update root value
    root->data = nodes[index++];

    constructBST(root->right, nodes, index);
}

// Function to convert a binary tree to a binary search tree
Node* binaryTreeToBST(Node* root) {
    vector<int> nodes;
    inorder(root, nodes);

    // sort the nodes
    sort(nodes.begin(), nodes.end());

    int index = 0;
    constructBST(root, nodes, index);
    return root;
}
```

```
// Function to print the inorder traversal of a binary tree
void printInorder(Node* root) {
    if (root == NULL) {
        return;
    }
    printInorder(root->left);
    cout << root->data << " ";
    printInorder(root->right);
}
```

```
Node* leftMost(Node* node) {  
    Node* curr = node;  
    while (curr->left != nullptr)  
        curr = curr->left;  
    return curr;  
}
```

```
Node* getSucc(Node* root, int target) {++++}
```

```
// Base Case 1: No Inorder Successor
```

```
if (root == nullptr)  
    return nullptr;
```

```
// Base Case 2: root is same as target and  
// right child is not empty
```

```
if (root->data == target && root->right != nullptr)  
    return leftMost(root->right);
```

```
// Use BST properties to search for  
// target and successor
```

```
Node* succ = nullptr;
```

```
Node* curr = root;
```

```
while (curr != nullptr) {
```

```
    // If curr node is greater, then it  
    // is a potential successor
```

```
    if (target < curr->data) {  
        succ = curr;  
        curr = curr->left;  
    }
```

```
    // If smaller, then successor must  
    // be in the right child
```

```
    else if (target >= curr->data)  
        curr = curr->right;
```

```
    // If same, the last visited  
    // greater value is the successor
```

```
    else  
        break;
```

```
}
```

```
return succ;
```

BST or not

// Function to find max value in the subtree

```
int maxValue(Node* node) {  
    if (node == nullptr)  
        return INT_MIN;  
    return max({ node->data,  
                maxValue(node->left),  
                maxValue(node->right) });  
}
```

```
int minValue(Node* node) {  
    if (node == nullptr)  
        return INT_MAX;  
    return min({ node->data,  
                minValue(node->left),  
                minValue(node->right) });  
}
```

```
bool isBST(Node* node) {  
    if (node == nullptr)  
        return true;  
  
    if (node->left != nullptr && maxValue(node->left) >= node->data)  
        return false;  
  
    // Check if the min of the right subtree  
    // is smaller than or equal to current node  
    if (node->right != nullptr && minValue(node->right) <= node->data)  
        return false;  
  
    // Recursively check if left and right subtrees are BSTs  
    return isBST(node->left) && isBST(node->right);  
}
```

Node with Kth largest element

```
// A function to find k'th largest element in a given tree.
void kthLargestUtil(Node* root, int k, int& c)
{
    // Base cases, the second condition is important to
    // avoid unnecessary recursive calls
    if (root == NULL || c >= k)
        return;

    // Follow reverse inorder traversal so that the
    // largest element is visited first
    kthLargestUtil(root->right, k, c);

    // Increment count of visited nodes
    c++;

    // If c becomes k now, then this is the k'th largest
    if (c == k)
    {
        cout << "K'th largest element is "
              << root->key << endl;
        return;
    }

    // Recur for left subtree
    kthLargestUtil(root->left, k, c);
}

// Function to find k'th largest element
void kthLargest(Node* root, int k)
{
    // Initialize count of nodes visited as 0
    int c = 0;

    // Note that c is passed by reference
    kthLargestUtil(root, k, c);
}

/* A utility function to insert a new node with given key in BST */
Node* insert(Node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->key)
        node->left = insert(node->left, key);
    else if (key > node->key)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}
```


Least common ancestor

```
// Function to find LCA of n1 and n2 assuming
// both are present in the BST
Node* lca(Node* root, int n1, int n2) {
    if (root == nullptr)
        return nullptr;

    // If both n1 and n2 are smaller than
    // root, go to left subtree
    if (root->data > n1 && root->data > n2)
        return lca(root->left, n1, n2);

    // If both n1 and n2 are greater than
    // root, go to right subtree
    if (root->data < n1 && root->data < n2)
        return lca(root->right, n1, n2);

    // If one n1 or n2 is on one side
    // and the other on the other side,
    // root is the LCA
    return root;
}
```

Median

```
struct Node* newNode(int item)
{
    struct Node* temp = new Node;
    temp->data = item;
    temp->left = temp->right = NULL;
    return temp;
}

/* A utility function to insert a new node with
given key in BST */
struct Node* insert(struct Node* node, int key)
{
    /* If the tree is empty, return a new node */
    if (node == NULL) return newNode(key);

    /* Otherwise, recur down the tree */
    if (key < node->data)
        node->left = insert(node->left, key);
    else if (key > node->data)
        node->right = insert(node->right, key);

    /* return the (unchanged) node pointer */
    return node;
}
```

```

int counNodes(struct Node* root)
{
    struct Node* current, * pre;

    // Initialise count of nodes as 0
    int count = 0;

    if (root == NULL)
        return count;

    current = root;
    while (current != NULL)
    {
        if (current->left == NULL)
        {
            // Count node if its left is NULL
            count++;

            // Move to its right
            current = current->right;
        }
        else
        {
            /* Find the inorder predecessor of current */
            pre = current->left;

            while (pre->right != NULL &&
                pre->right != current)
                pre = pre->right;

            /* Make current as right child of its
            inorder predecessor */
            if (pre->right == NULL)
            {
                pre->right = current;
                current = current->left;
            }

            /* Revert the changes made in if part to
            restore the original tree i.e., fix
            the right child of predecessor */
            else
            {
                pre->right = NULL;

                // Increment count if the current
                // node is to be visited
                count++;
                current = current->right;
            } /* End of if condition pre->right == NULL */
        } /* End of if condition current->left == NULL */
    } /* End of while */

    return count;
}

```

```

using Morris Inorder Traversal*/
int findMedian(struct Node* root)
{
    if (root == NULL)
        return 0;

    int count = counNodes(root);
    int currCount = 0;
    struct Node* current = root, * pre, * prev;

    while (current != NULL)
    {
        if (current->left == NULL)
        {
            // count current node
            currCount++;

            // check if current node is the median
            // Odd case
            if (count % 2 != 0 && currCount == (count + 1) / 2)
                return current->data;

            // Even case
            else if (count % 2 == 0 && currCount == (count / 2) + 1)
                return (prev->data + current->data) / 2;

            // Update prev for even no. of nodes
            prev = current;

            // Move to the right
            current = current->right;
        }
        else
        {
            /* Find the inorder predecessor of current */
            pre = current->left;
            while (pre->right != NULL && pre->right != current)
                pre = pre->right;

            /* Make current as right child of its inorder predecessor */
            if (pre->right == NULL)
            {
                pre->right = current;
                current = current->left;
            }

            /* Revert the changes made in if part to restore the original
            tree i.e., fix the right child of predecessor */

```



```

/* Revert the changes made in if part to restore the original
tree i.e., fix the right child of predecessor */
else
{
    pre->right = NULL;

    prev = pre;

    // Count current node
    currCount++;

    // Check if the current node is the median
    if (count % 2 != 0 && currCount == (count + 1) / 2)
        return current->data;

    else if (count % 2 == 0 && currCount == (count / 2) + 1)
        return (prev->data + current->data) / 2;

    // update prev node for the case of even
    // no. of nodes
    prev = current;
    current = current->right;
} /* End of if condition pre->right == NULL */
} /* End of if condition current->left == NULL */
} /* End of while */
}

```

Sum of two vals equal to a num

```
✓ bool findVal(Node* root, int x) {  
    if (root == nullptr) return false;  
  
    if (root->data == x) return true;  
    else if (root->data < x)  
        return findVal(root->right, x);  
    else  
        return findVal(root->left, x);  
}  
  
✓ int countPairs(Node* root1, Node* root2, int x) {  
    if (root1 == nullptr) return 0;  
  
    int ans = 0;  
  
    // If pair (root1.data, x-root1.data) exists,  
    // then increment the ans.  
    if (findVal(root2, x - root1->data))  
        ans++;  
  
    // Recursively check for left and right subtree.  
    ans += countPairs(root1->left, root2, x);  
    ans += countPairs(root1->right, root2, x);  
  
    return ans;  
}
```